Technik der MMI, UXD-B
Praktikum, WiSe 2026/27

Praktikumsblatt 6
Prof. Dr. Andreas Riener, Prof. Dr. Simon Nestler

Face- und Fiducialtracking

Worum geht es in diesem Blatt?

Sie arbeiten mit kamerabasierter Erkennung in vvvv gamma und der Bibliothek VL.OpenCV. Sie lassen Gesichter in einem Video und im Kamera-Livestream finden und verfolgen anschließend ArUco-Marker: gedruckte quadratische Muster, die sich zuverlässiger erkennen lassen als ein Gesicht. Zum Schluss messen Sie mit zwei Markern einen Abstand.

Was erwarten wir von der Dokumentation (Protokoll)?

- Beantworten Sie die gestellten Fragen in einem Protokoll (Moodle-Fragebogen/Quiz)
- Machen Sie zusätzlich Screenshots Ihrer Patches/Programme und Fotos, die Ihre Lösungen in „Aktion“ zeigen (vvvv-Screenshots erstellen Sie direkt in vvvv mit der Tastenkombination „**STRG + 2**“); vergessen Sie nicht, die Patches zu speichern!
- Halten Sie Ihre **eigene Interpretation** fest: Was haben Sie im Experiment erkannt, was ist Ihnen aufgefallen?
- Welche **Probleme** sind Ihnen begegnet, und wie haben Sie sie **gelöst**?

Wie?

- **Bearbeiten Sie zur Abgabe den Test „Abgabe Praktikumsblatt 6“** in Moodle (Kursraum Technik der Mensch-Maschine Interaktion).
- **Kopieren Sie dazu Ihre Antworten, Screenshots, Fotos usw. in die jeweiligen Antwortfelder.** Das Protokoll können Sie in der Praktikumsstunde direkt im Angabe-Worddokument oder in einem separaten Textdokument schreiben.
- **Speichern Sie die vvvv-Patches und geben Sie sie als Anhänge mit ab!**
- In Zweiergruppen lädt nur die protokollführende Person hoch; im Protokoll stehen beide mit vollständigem Namen.
- **ACHTUNG: Chromium-basierte Browser können offenbar keine Bilder kopieren. Nutzen Sie in dem Fall einen anderen Browser, z. B. Firefox.**

Abgabe?

- **Abgabe innerhalb von 48 Stunden** nach Ihrem Praktikumstermin, in Moodle

In diesem Aufgabenblatt dreht sich alles um visuelle Datenverarbeitung. Mit Facetracking/-detection erkennen Sie Merkmale des menschlichen Gesichts in einem Video bzw. Kamera-Livestream, und mit sogenannten „*ArUco*“-Markern sehen Sie an einem einfachen Beispiel, wie markerbasiertes Tracking funktioniert. „*ArUco*“-Marker nutzt man häufig in „*Augmented Reality*“-Applikationen.

1) Gesichtserkennung in einem Video bzw. Kamera-Livestream

In dieser ersten Aufgabe erkennen Sie mit „*vvvv*“ Position und Größe eines Gesichts in einem Video bzw. im Livestream einer Webcam. Es geht um Gesichtsdetektion, d. h. darum, Gesichter oder Gesichtsfeatures in einem Bild zu finden und zu kennzeichnen. Mit passenden Analysekomponenten ließen sich sogar Alter oder Geschlecht aus dem Video-/Kamerabild ablesen oder mehrere Personen in einem Video unterscheiden. Facetracking bzw. -detection als „Eingabegerät“ gehört zu den natürlichen Benutzerschnittstellen („Natural User Interfaces“), weil man für die Interaktion keine zusätzlichen Gegenstände braucht. Für die Beispiele dieses Aufgabenblatts verwenden wir eine Webcam.

Sie verwenden drei sogenannte „*Haar-Feature*“-Trainingsdateien (Infos dazu: `https://en.wikipedia.org/wiki/Haar-like_features`), um z. B. den Oberkörper, das Gesicht oder die Augen in einem Bild zu finden.

Die Computer-Vision-Library VL.OpenCV müssen Sie vor der ersten Nutzung über die „Commandline“ installieren:

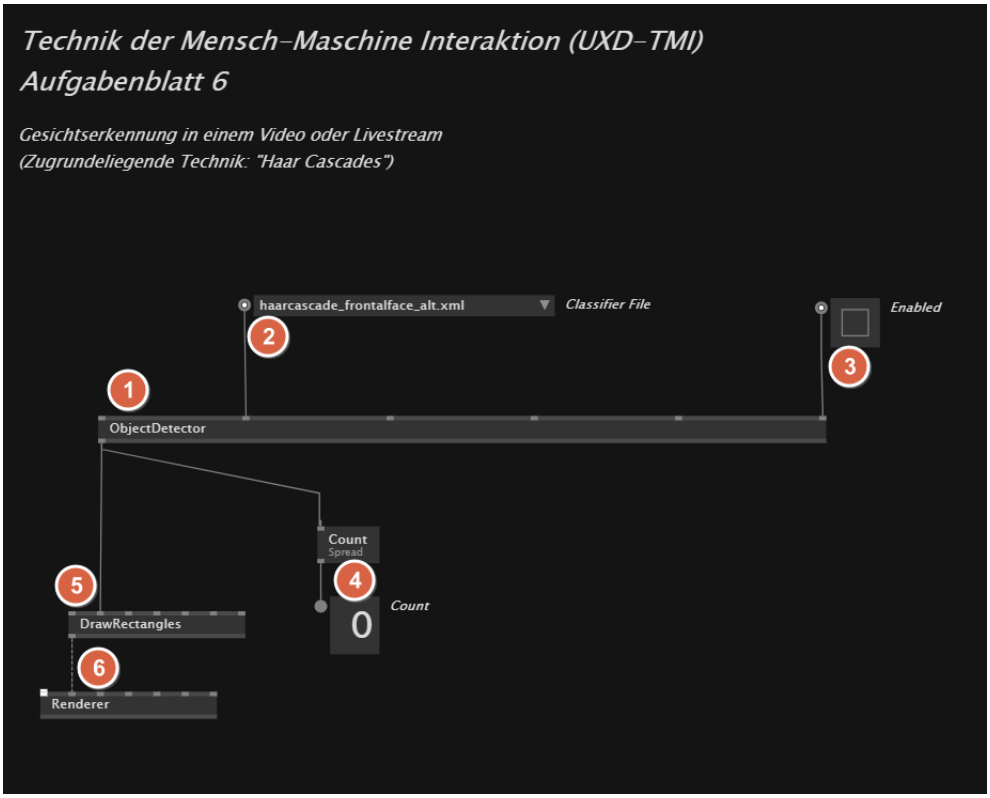
```
Quad-Menü → Manage Nugets → Commandline: nuget install VL.OpenCV
```

Im NodeBrowser binden Sie die Library dann mit einem Klick auf das Paket VL.OpenCV ein.

Schritt 1: Die zentrale „*vvvv*“-Komponente, um Gesichter (oder andere Features) in einem Bild oder Video zu erkennen, ist der „`ObjectDetector [OpenCV.Detection]`“-Node. Ihm übergeben Sie ein Bild, eine Videodatei oder einen Kamera-Livestream (Eingangs-Pin „Input“). Über den Eingangs-Pin „Classifier File“ legen Sie fest, welches Feature bzw. welche Region im Bild erkannt werden soll (z. B. Gesicht, Augen, Mundpartie). Die weiteren Eingabeparameter („Scale Factor“, „Min Neighbors“, „Scale Cascade“, „Flags“) dienen dem Feintuning; sie kommen weiter unten genauer dran.

Vorgangsweise

- Legen Sie einen neuen, leeren Patch an
- Bauen Sie das Programm (d. h. den „Patch“) nach und nach wie unten dargestellt auf:
  - **Hinweis: Die Nummern in Klammern entsprechen den Zahlen in den Kreisen der Patch-Screenshots!**



(1) Einen „`ObjectDetector [OpenCV.Detection]`“-Node in den Patch einfügen

- Sehen Sie sich dazu den Hilfe-Patch zu „`ObjectDetector`“ an (F1-Taste)
- Sehen Sie sich auch die Dokumentation zu OpenCV im Help Browser an (Help Browser → Learn → Documentation → OpenCV)

(2) Eine „IOBox“ am Eingangs-Pin „Classifier File“ des „`ObjectDetector [OpenCV.Detection]`“-Nodes erzeugen. Über diese IOBox laden Sie (etwas später) eine bestimmte „Trainingsdatei für *Haar Cascades*“, hier für Kopf-/Schultererkennung, Gesichtserkennung bzw. Augenerkennung.

(3) Eine „IOBox“, Datentyp „`Boolean`“, Behavior „`Toggle`“, am Eingang „Enabled“ erzeugen. Mit diesem „Schalter“ starten Sie später die Objekterkennung.

(4) Fügen Sie einen „`Count [Collections.Spread]`“-Node ein und verbinden Sie ihn mit dem Ausgangs-Pin „Rectangles“ des „*ObjectDetector*“-Nodes. Der Count-Node ist sehr oft nützlich: Er gibt die Anzahl der „Slices“ in einem „Spread“ zurück und entspricht in Java dem `.length`-Operator von Arrays. Hier zählt bzw. zeigt er, wie viele Personen bzw. Objekte in einem Bild bzw. Video erkannt wurden. Fügen Sie außerdem am Ausgang des Count-Nodes einen „IOBox“-Node ein und stellen Sie seine Schriftgröße (Parameter „Font Size“) auf „`20,0`“ px. Dieses Feld zeigt dann an, wie viele Objekte der „*ObjectDetector*“-Node erkannt hat.

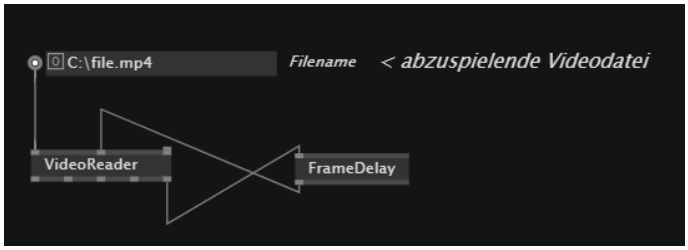
(5) Einen „`DrawRectangles [OpenCV.Drawing]`“-Node einfügen und an den Ausgangs-Pin des „*ObjectDetector*“-Nodes anschließen. Dieser Node zeichnet ein Rechteck an die Stelle, an der der „*ObjectDetector*“-Node ein Objekt erkannt hat.

(6) Fügen Sie noch einen „`Renderer [OpenCV.Sink]`“-Node ein. Er zeigt die Bild-/Videoquelle an, die Sie als Nächstes an den „*ObjectDetector*“-Node übergeben.

Damit ist die „Logik“ für das Objekttracking fertig. Als Nächstes geht es um die Eingabe: Bild, Videodatei bzw. Kamera-Livestream. Speichern Sie Ihren Patch als „`FacedetectionV1.vl`“ ab.

Schritt 2: Zuerst erkennen Sie Features in einer vorab aufgezeichneten Videodatei (später schließen Sie eine Webcam an und nutzen den Livestream der Kamera). Videos öffnen und abspielen Sie mit dem „`VideoReader [OpenCV.Source]`“-Node. Er spielt universell Videodateien ab.

Damit das Video am Ende automatisch wieder startet, verbinden Sie den Ausgang „Finished“ über einen „`FrameDelay`“-Node mit dem Eingang „Seek“:



Vorgangsweise

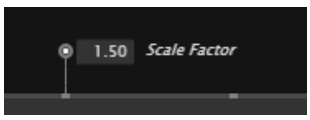
- Verschieben Sie den bisherigen Teil des Patches (rund um den „*ObjectDetector*“-Node) nach unten, damit „oben“ Platz für den „*VideoReader*“-Node ist.
- Erweitern Sie den Patch nach und nach wie in der Abbildung auf der nächsten Seite:

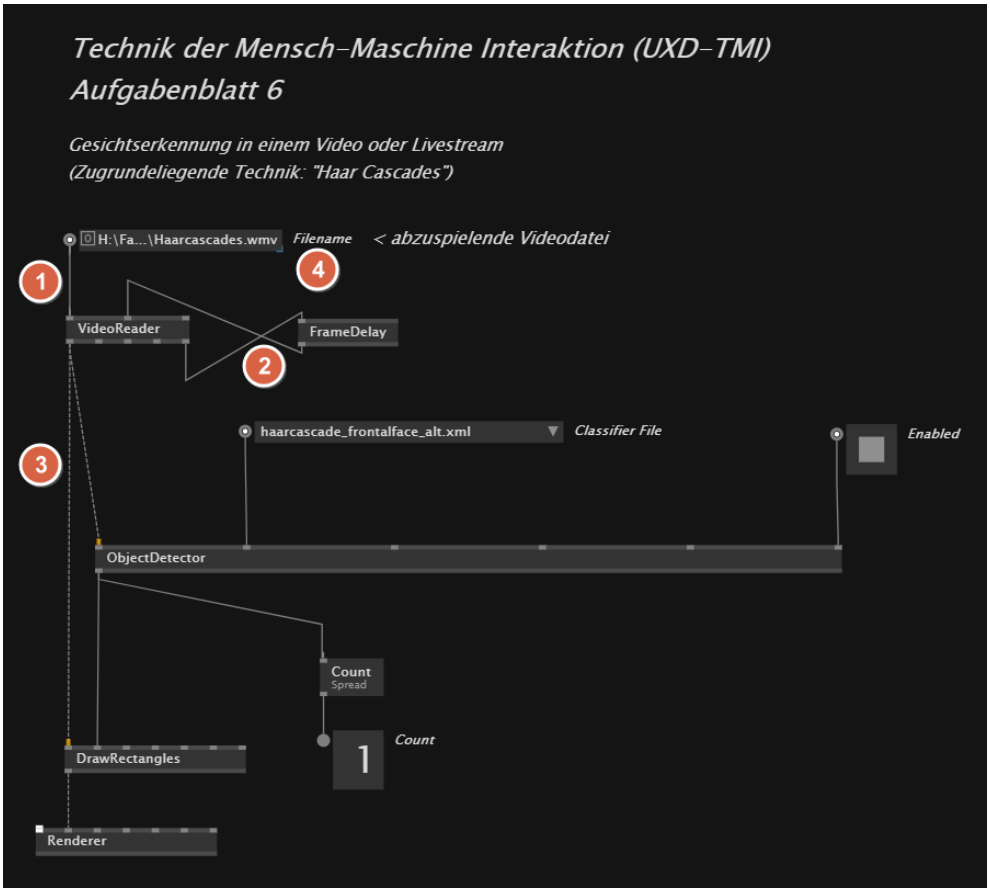
(1) Einen „`VideoReader [OpenCV.Source]`“-Node in den Patch einfügen und am Eingang „Filename“ eine IOBox erzeugen.

(2) Einen „`FrameDelay [Control]`“-Node einfügen und den Ausgang „Finished“ wie dargestellt mit dem Eingang „Seek“ verbinden, damit sich das Video immer wieder wiederholt (=„*Loop*“).

(3) Verbinden Sie den Ausgang „Output“ des „*VideoReader*“-Nodes sowohl mit dem Eingang „Input“ des „*ObjectDetector*“-Nodes als auch mit dem Input von „*DrawRectangles*“. Fügen Sie einen „`IOBox (String)`“-Node ein und verbinden Sie ihn mit dem ersten Eingangs-Pin „Filename“ des „`VideoReader [OpenCV.Source]`“-Nodes. Mit diesem Node laden Sie das Video für die Objekterkennung.

(4) Öffnen Sie zum Schluss die IOBox am „*VideoReader*“ mit der rechten Maustaste. Es sollte sich ein Explorer-Fenster öffnen, in dem Sie die Datei „`Haarcascades.wmv`“ auswählen.





Damit ist auch dieser Teil fertig, und das Video sollte bereits im Renderer-Fenster laufen…

Schritt 3: Zum Schluss wählen Sie noch die Trainingsdateien für das Objekttracking am „`ObjectDetector [OpenCV.Detection]`“-Node aus.

Vorgangsweise

- Klicken Sie mit der rechten Maustaste auf den Node mit der Bezeichnung „Classifier File“ (eine „IOBox“) und wählen Sie z. B. „***haarcascade_frontalface_alt.xml****“ aus.
- Hängen Sie zusätzlich eine „`IOBox [Float32]`“ an den Input-Pin „Scale Factor“ des „*ObjectDetector*“-Nodes, um den Tracking-Algorithmus zu konfigurieren.
- Speichern Sie Ihren Patch als „`FacedetectionV2.vl`“ ab.

…im Renderer-Fenster sollte jetzt das Gesicht „getrackt“ werden.

Frage 1: a) Was stellen Sie beim Betrachten des Videos (was das Tracking angeht) generell fest? b) Was verändert sich, wenn Sie den Wert für „Scale Factor“ ändern (z. B. 0.1, 1, 10, 100)?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Frage 2: Was bedeutet es, wenn der rechteckige, blaue Bereich aus dem Bild verschwindet?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Frage 3: Was stellen Sie fest, wenn Sie jetzt den Wert des Parameters „Min Neighbors“ auf andere Werte ändern (direkt im Inspector oder über eine IOBox)? Was ist der Grund für das Verhalten, das Sie beobachten?

Sehen Sie sich jetzt noch die anderen Tracking-Varianten an. Öffnen Sie dazu als „Classifier File“ zuerst die Datei „***haarcascade_eye.xml**“ *und schließlich* „**haarcascade_smile.xml***“.

Frage 4: Welche Veränderung stellen Sie jetzt fest?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Schritt 4: Als Nächstes verbinden Sie „*vvvv*“ mit einer Webcam und erkennen und „tracken“ im Kamera-Livestream Objekte (wieder a) Augen, b) Kopf+Schulter, c) Gesicht oder d) Mund).

Vorgangsweise

- Beginnen Sie mit dem bereits geöffneten Patch und speichern Sie ihn als „`FacedetectionV3.vl`“.
- Erweitern Sie diesen Patch so:

(1) Einen „`VideoIn [OpenCV.Source]`“-Node in den Patch einfügen und so konfigurieren:

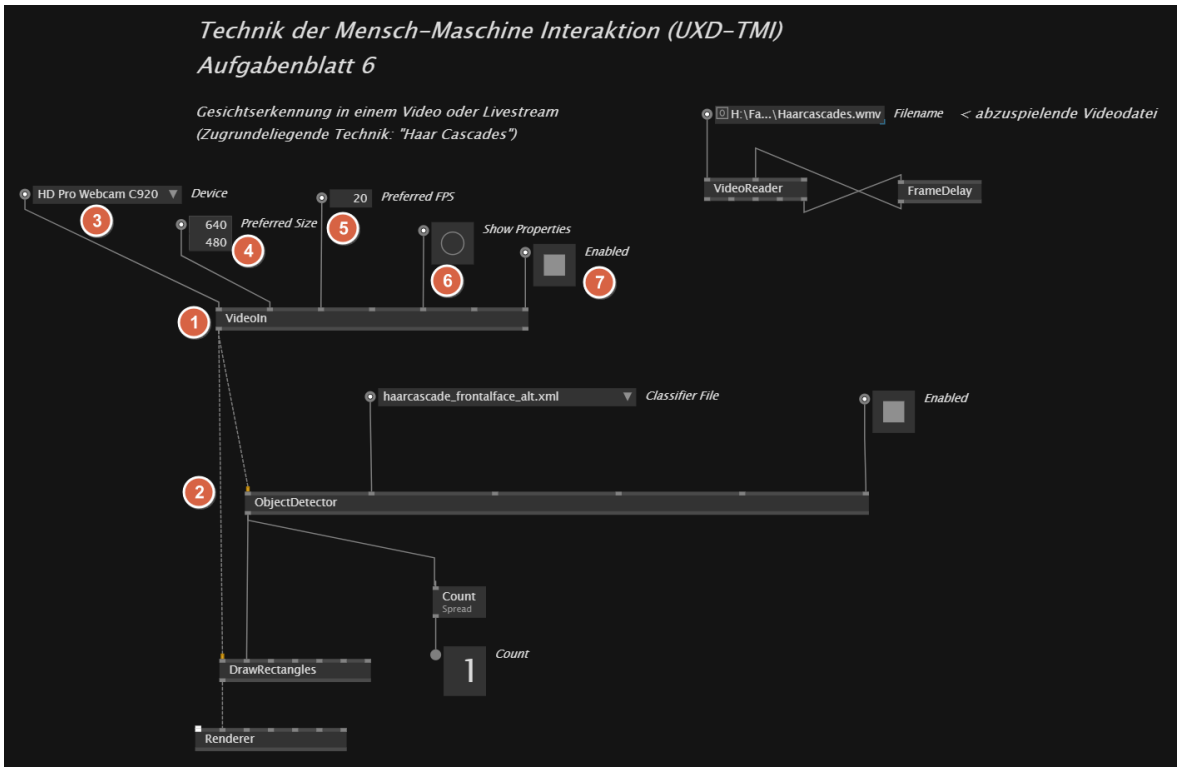
(2) Verbinden Sie den Output-Pin des „*VideoIn*“-Nodes sowohl mit dem Input des „*ObjectDetector*“-Nodes als auch mit dem Input des „*DrawRectangles*“-Nodes.

(3) Erzeugen Sie eine „IOBox“ am Eingangs-Pin „Device“ des „*VideoIn*“-Nodes. Der Eintrag „Default“ bezieht sich auf die erste Kamera, die das System erkennt. Sie können aber auch die Webcam „HD Pro Webcam C920“ auswählen.

(4) und (5) Erzeugen Sie weitere „IOBox“-Nodes an den Eingangs-Pins „Preferred Size“ und „Preferred FPS“. Damit geben Sie die bevorzugte Bildwiederholrate der Kamera (Frames per second, auch „Framerate“ genannt), die Bildbreite und die Bildhöhe an. Die Default-Framerate ist 30 FPS und die Default-Bildgröße 1280x720. Testen Sie eine Bildgröße von 640x480 Pixel und eine Framerate von 20 FPS.

(6) Erzeugen Sie eine „IOBox“, Behavior: „`Bang`“, am Input „Show Properties“. Ein Rechtsklick auf diesen Node öffnet die Eigenschaften-Seite der jeweils aktiven Kamera. Probieren Sie das ruhig aus!

(7) Erzeugen Sie eine „IOBox“, Behavior „`Toggle`“, am letzten Eingangs-Pin „Enabled“. Über diese IOBox schalten Sie die Kamera ein und aus.



Damit ist die Konfiguration bzw. Integration der Webcam komplett. Speichern Sie den Patch ab und aktivieren Sie die Kamera („*VideoIn*“) und die Gesichtserkennung („*ObjectDetector*“).

Frage 5: Wie gut funktioniert das Tracking mit dem Live-Bild im Vergleich zum Video von oben? Verwenden Sie auch hier die Trainingsdateien für Gesichtstracking, Oberkörpertracking bzw. Mundtracking.

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

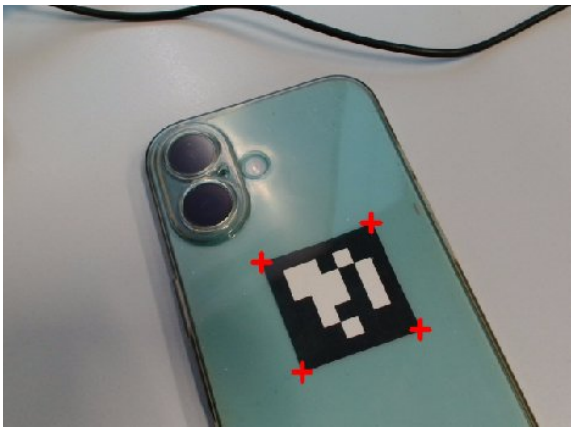
Frage 6: Experimentieren Sie jetzt mit mehreren Personen. Wie viele Personen schafft der „*ObjectDetector*“-Node gleichzeitig? Beobachten Sie den Zähler mit der Bezeichnung „Count“ und die Anzahl der Rechtecke im Videobild. Was fällt Ihnen sonst noch auf?

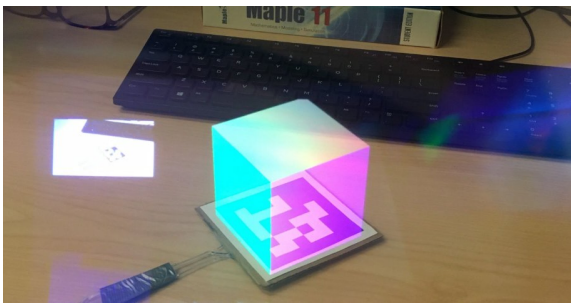
Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

2) Markerbasiertes Tracking (mit „ArUco“-Markern)

„*ArUco*“-Marker sind bekannte Vertreter des sog. markerbasierten Trackings. Die quadratischen Muster lassen sich mit einer einfachen Webcam erfassen und verarbeiten. „*ArUco*“-Marker kann man ausdrucken und auf jede beliebige Oberfläche kleben. In interaktiven Anwendungen sitzen die Marker oft auf der Unterseite von Objekten (bei interaktiven Tischen werden sie von unten abgefilmt) und sind so für den Benutzer nicht sichtbar.









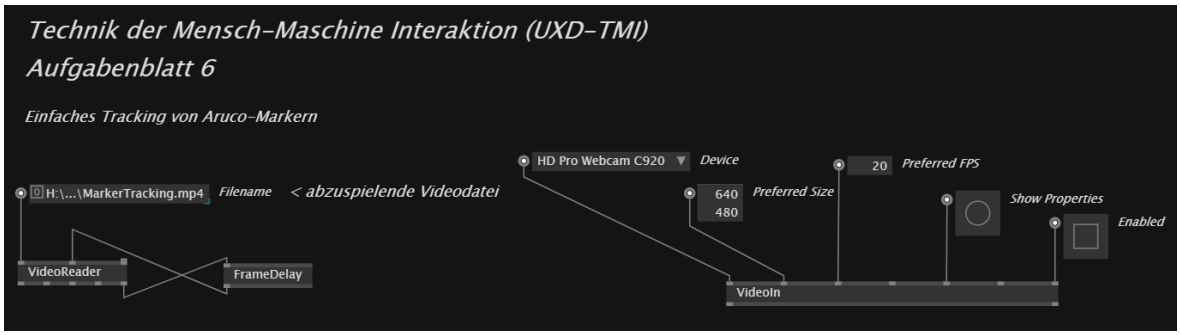
Beispiele für ArUco-Marker (erstes Bild); ArUco-Marker im Einsatz (übrige Bilder).

Mit „*ArUco*“-Markern identifizieren Sie beliebige Objekte sehr einfach und zuverlässig in einem Kamera-Livestream. Schon mit mehreren solcher Marker lassen sich erste zusammenhängende Interaktionen entwerfen, z. B. indem man unterschiedliche Objekte verknüpft (Größe, Distanz zueinander usw.). So erwecken Sie „leblose“ Gegenstände im Kamerabild (und dann in einer Applikation) sehr einfach zum Leben…

Schritt 1: Eine Möglichkeit, beliebige Objekte in einem Live-Kamerabild zu tracken, ist ArUco-Tracking. Die zentrale „*vvvv*“-Komponente, um ArUco-Marker einzulesen und zu verarbeiten, ist der „`MarkerDetector [OpenCV.Detection]`“-Node. Er erkennt ArUco-Marker im übergebenen Kamerabild (oder Videostream) und liefert für jeden Marker die Position der Eckpunkte des Markers in einem Spread. Wie mächtig das Konzept ist, zeigt eine einfache Anwendung: Der Renderer zeigt ein Kamera-Livebild bzw. Video als Hintergrund an, der „`MarkerDetector [OpenCV.Detection]`“-Node erkennt die Marker im Bild/Video, und Kreuzchen an den Ecken der Marker werden dem Kamera-/Videobild überlagert.

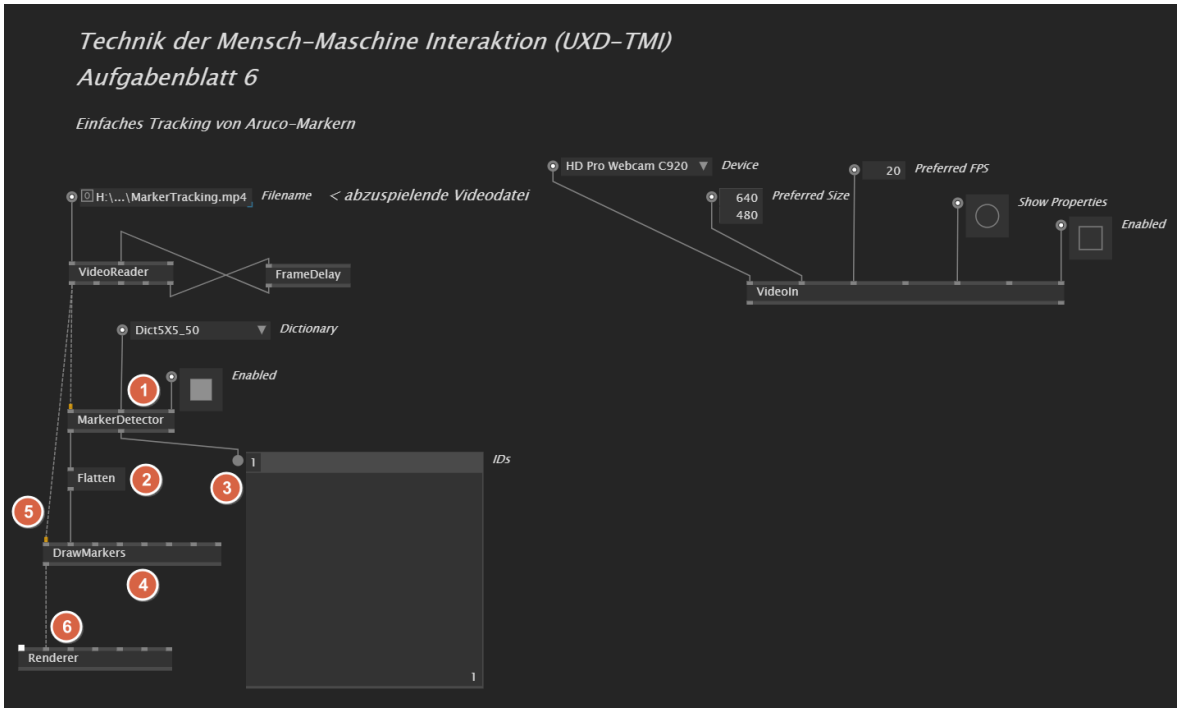
Vorgangsweise

– Beginnen Sie mit dem Patch „`FacedetectionV3.vl`“ von vorhin und lassen Sie nur die beiden Komponenten „*VideoReader*“ und „*VideoIn*“ mit allen Konfigurationsparametern stehen. Den Rest (d. h. alles „unterhalb“ des „*VideoIn*“-Nodes) entfernen Sie aus dem Patch. Das Ergebnis sollte etwa so aussehen; speichern Sie das Zwischenergebnis als Datei „`MarkerTrackingV1.vl`“ ab.



– Konfigurieren Sie den „*VideoReader*“-Node so (siehe auch Skizze): Öffnen Sie als Video (mit Rechtsklick auf „Filename“) „***MarkerTracking.mp4***“.

– Bauen Sie den Patch dann nach und nach wie in der Skizze unten auf.



(1) Als Erstes fügen Sie einen „`MarkerDetector [OpenCV.Detection]`“-Node ein und verbinden ihn mit dem Ausgangs-Pin des „*VideoReader*“-Nodes. Außerdem erzeugen Sie eine IOBox am Eingangs-Pin „Dictionary“ und eine IOBox (Behavior: „`Toggle`“) am Pin „Enabled“. Wählen Sie im Listenfeld am Pin „Dictionary“ „`Dict5X5_50`“ aus.

(2) Fügen Sie einen „`Flatten [Collections.Spread]`“-Node ein und verbinden Sie seinen „Input“ mit dem Ausgangs-Pin „Markers“ des „`MarkerDetector [OpenCV.Detection]`“-Nodes. Diese Komponente „entfernt“ eine Dimension eines mehrdimensionalen Spreads.

(3) Erzeugen Sie eine IOBox am Ausgang „IDs“ des „*MarkerDetector*“-Nodes.

(4) bis (5) Fügen Sie einen „`DrawMarkers [OpenCV.Drawing]`“-Node ein. Verbinden Sie seine Eingangs-Pins „Input“ und „Markers“ mit dem Output des „*VideoReader*“- bzw. des „*Flatten*“-Nodes.

(6) Fügen Sie einen „`Renderer [OpenCV.Sink]`“-Node ein und verbinden Sie seinen Eingang „Input“ mit dem „Output“ von „*DrawMarkers*“.

Als Ergebnis sollte jetzt die Videodatei im Renderer-Fenster laufen. Damit ist der Patch fertig (vier Kreuze sollten den Ecken der Marker im Video folgen).

Im nächsten Schritt testen Sie das interaktiv mit dem Kamera-Livebild und eigenen ArUco-Markern. Experimentieren Sie dazu mit dem fertigen Patch und einem Kamera-Livebild (statt des Videos schließen Sie die Kamera an „*MarkerDetector*“ und „*DrawMarkers*“ an).

Frage 7: Wie lassen sich mehrere Marker in vvvv voneinander unterscheiden? Wie finden Sie heraus, welche ID zu welchem Marker gehört?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Frage 8: Was stellen Sie fest, wenn Sie das ganze A4-Blatt mit den 5x7 ArUco-Markern unter die Kamera halten? Was passiert, wenn Sie das Blatt drehen/rotieren?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

Frage 9: Schneiden Sie jetzt einige Marker aus dem A4-Blatt aus und kleben Sie sie (z. B. mit Tesa-Klebeband) auf Alltagsgegenstände (z. B. Ihr Smartphone). Was beobachten Sie, bzw. wofür könnten Sie die Marker in realen Anwendungen nutzen?

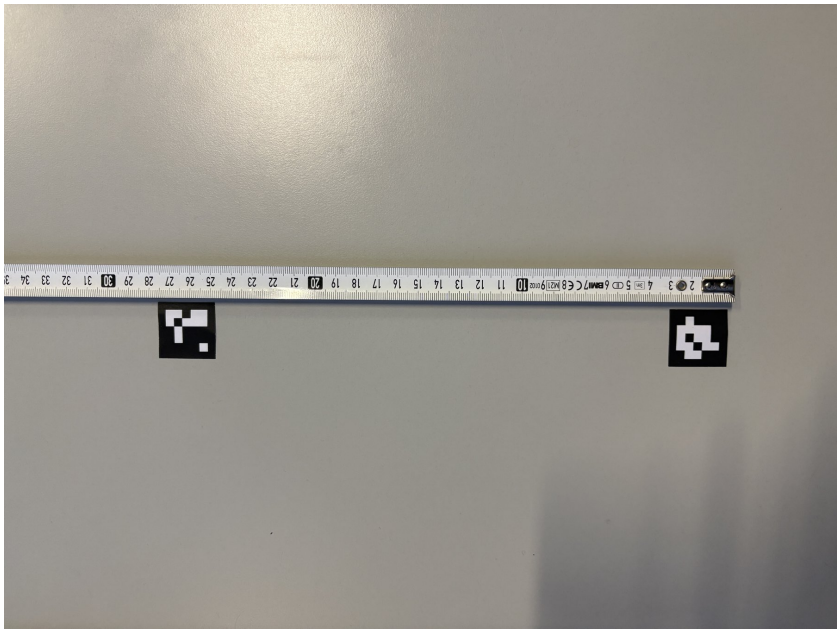
Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!

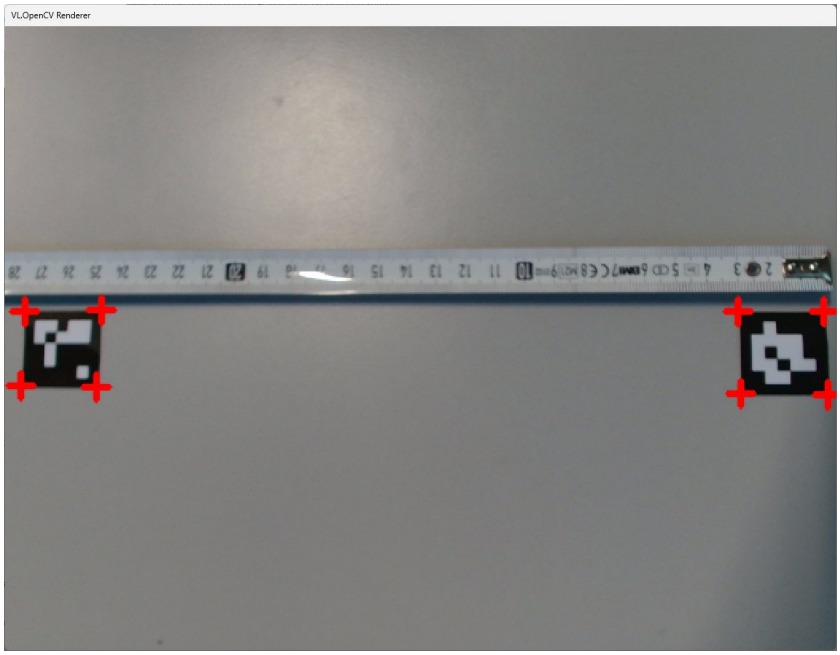
Praxisbeispiel: Distanzmessung

Zum Abschluss implementieren Sie mit 2 Markern ein einfaches System, das Abstände misst (siehe unten). Speichern Sie Ihren Patch zuerst als „`MarkerTrackingV2.vl`“ ab. Was müssen Sie ändern, damit eine IOBox in Ihrem Patch den Abstand zwischen zwei ArUco-Markern in Millimetern oder Zentimetern ausgibt?

Hinweis: Für die Abstandsberechnung können Sie diese Formel verwenden; (x_1, y_1) ist die Position des 1. Markers, (x_2, y_2) die Position des 2. Markers.

$d = \sqrt{((x_2 - x_1)^2 + (y_2 - y_1)^2)}$





Frage 10: Dokumentieren Sie die nötigen Änderungen in Ihrem Patch (Text, Screenshot). Welche Koordinaten liefert der „*MarkerDetector*“-Node für jeden erkannten Marker, wo ist der Koordinatenursprung der Marker? Funktioniert die Abstandsmessung korrekt (wann bzw. warum nicht)? Wie ließe sich der Patch verbessern? Welche Probleme erkennen Sie, wenn Sie eine (allgemeine) Messung vornehmen möchten?

Dokumentieren Sie Ihre Erkenntnisse in Ihrem Protokoll (Moodle-Quiz)!